

Búsqueda Heurística (2a. parte)

Javier Béjar, Javier Vázquez

Algoritmos Básicos de la Inteligencia Artificial - 2022/2023 1Q

CS - GIA



Búsqueda Informada

- ⦿ Supone la existencia de una función de evaluación que se utiliza para guiar el proceso de búsqueda
- ⦿ En cada momento se selecciona el estado o las operaciones más prometedores según esa función

2

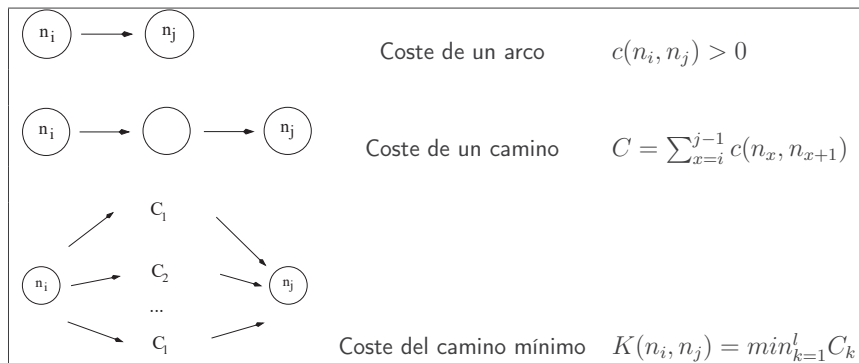
- ⦿ Algoritmos que trabajan en el espacio de estados:
 - Branch and Bound, Best First Search
 - A*, IDA*, MA*, ...
- ⦿ Algoritmos que trabajan en el espacio de soluciones:
 - Hill climbing
 - Simulated annealing
 - Algoritmos Genéticos

3

Búsqueda Heurística guiada por G y H

Búsqueda Heurística guiada por dos estimadores (G y H)

- ⦿ Supone la existencia de una función de evaluación que combina
 - la estimación del coste mínimo acumulado desde un estado n del espacio de estados al estado inicial ($g(n)$)
 - la estimación de la distancia desde un estado n del espacio de estados al (a un) objetivo ($h(n)$)
- ⦿ La función se utiliza para guiar el proceso haciendo que en cada momento se seleccione el estado o las operaciones más prometedores

Si n_j es un nodo terminal

$$h^*(n_i) = K(n_i, n_j)$$

Si n_i es un nodo inicial

$$g^*(n_j) = K(n_i, n_j)$$

Si existen varios nodos terminales $T = \{t_1, \dots, t_l\}$

$$h^*(n_i) = \min_{k=1}^l K(n_i, t_k)$$

Si existen varios nodos iniciales $S = \{s_1, \dots, s_l\}$

$$g^*(n_j) = \min_{k=1}^l K(s_k, n_j)$$

6

La función de evaluación tiene dos componentes:

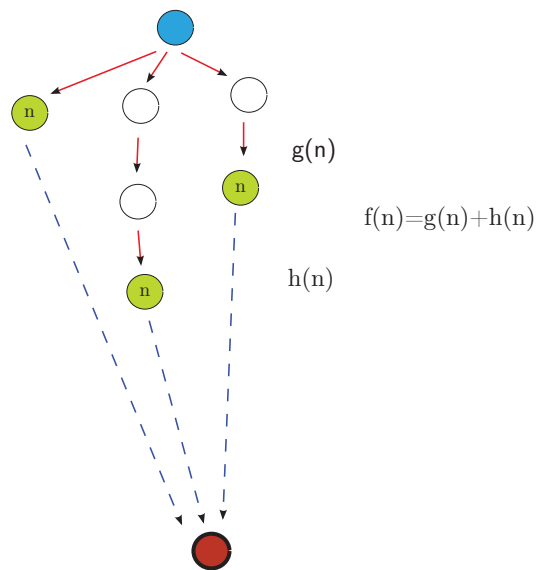
1. coste para ir desde el (un) inicio al nodo actual
2. coste (estimado) para ir desde el nodo actual a una solución

$$f(n) = g(n) + h(n)$$

- ⊙ f es un valor **estimado** del coste total del camino que pasa por n
- ⊙ h (heurístico) es un valor **estimado** de lo que falta para llegar desde n al (a un) objetivo
- ⊙ g es un coste **estimado** (lo gastado por el camino más corto **conocido** hasta n)

La **preferencia** es siempre del nodo con menor f , en caso de empate, la preferencia es del nodo con menor h .

7



Con esta función podemos variar el comportamiento del algoritmo

- ⊙ Si $\forall n \ h(n) = 0$, todo estará controlado por g (estaremos en presencia de un algoritmo de Branch & Bound)
- ⊙ Si $\forall n \ h(n) = 0$ y además el coste de todos los arcos es 1 estaremos realizando una búsqueda en anchura. Si dicho coste fuera 0, la búsqueda sería aleatoria
- ⊙ Al ser h una estimación del verdadero coste h^* , cuanto más se aproxime h a h^* mayor será la tendencia a explorar en profundidad. Si $h = h^*$ entonces A* converge directamente hacia el objetivo

Se puede demostrar que si $h(n)$ es un minorante del coste real $h^*(n)$, es decir si $\forall n \ h(n) \leq h^*(n)$ A* encontrará (de haberlo) un camino óptimo.

Algoritmo: A*

```
Est_abiertos.insertar(Estado inicial)
```

```
Actual ← Est_abiertos.primer()
```

```
mientras no es_final?(Actual) y no Est_abiertos.vací() hacer
```

```
    Est_abiertos.borrar_primer()
```

```
    Est_cerrados.insertar(Actual)
```

```
    hijos ← generar_sucesores (Actual)
```

```
    hijos ← tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)
```

```
    Est_abiertos.insertar(Hijos)
```

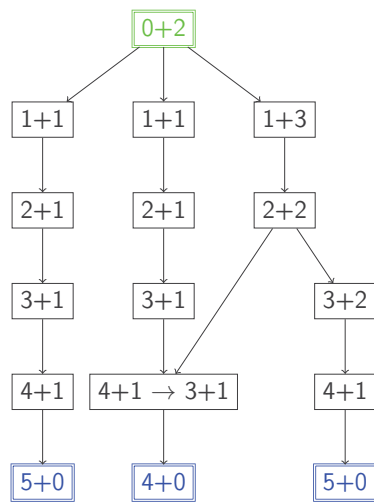
```
    Actual ← Est_abiertos.primer()
```

- ⊙ La estructura de abiertos es una cola con prioridad
- ⊙ La prioridad la marca la función de estimación ($f(n) = g(n) + h(n)$)
- ⊙ En cada iteración se escoge el mejor camino (el primero de la cola)
- ⊙ ¡Es el mismo algoritmo que el Best First!

10

- ⊙ Si es un repetido que está en la estructura de abiertos
 - Si su coste es menor substituímos el coste por el nuevo, esto podrá variar su posición en la estructura de abiertos
 - Si su coste es igual o mayor nos olvidamos del nodo
- ⊙ Si es un repetido que esta en la estructura de cerrados
 - Si su coste es menor reabrimos el nodo insertándolo en la estructura de abiertos con el nuevo coste ¡Atención! No hacemos nada con sus sucesores, ya se reabrirán si hace falta
 - Si su coste es mayor o igual nos olvidamos del nodo

11

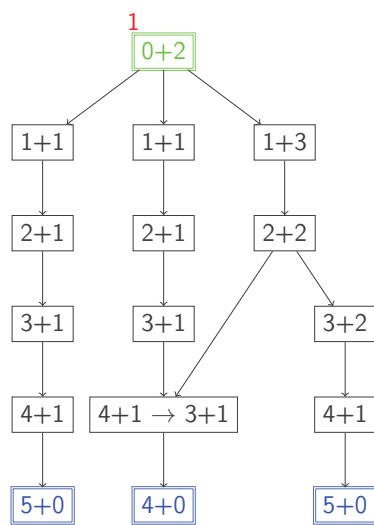


Abiertos

0+2

Cerrados

12



Abiertos

1+1

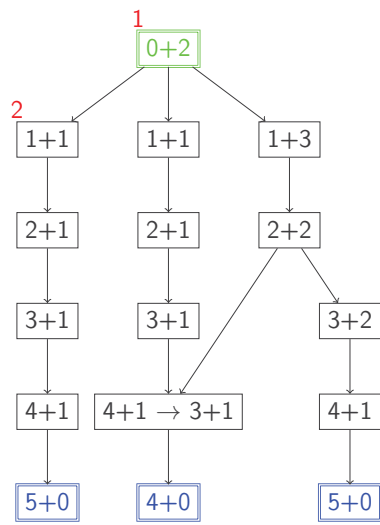
1+1

1+3

Cerrados

0+2

12

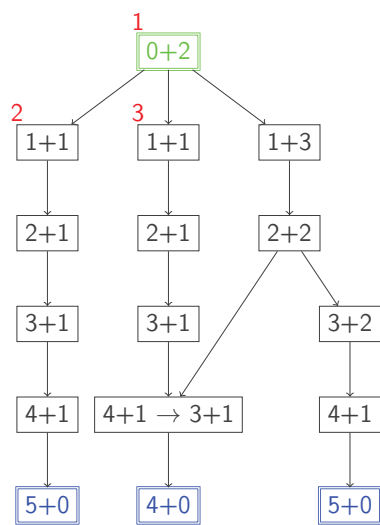


Abiertos

$1+1$
 $2+1$
 $1+3$

Cerrados

$0+2$
 $1+1$

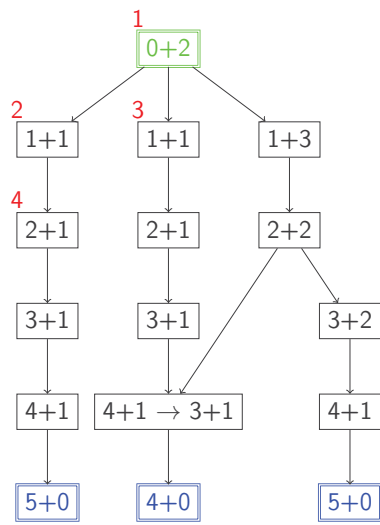


Abiertos

$2+1$
 $2+1$
 $1+3$

Cerrados

$0+2$
 $1+1$
 $1+1$



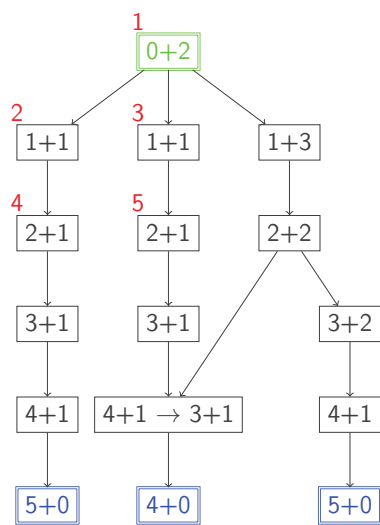
Abiertos

2+1
3+1
1+3

Cerrados

0+2
1+1
1+1
2+1

12



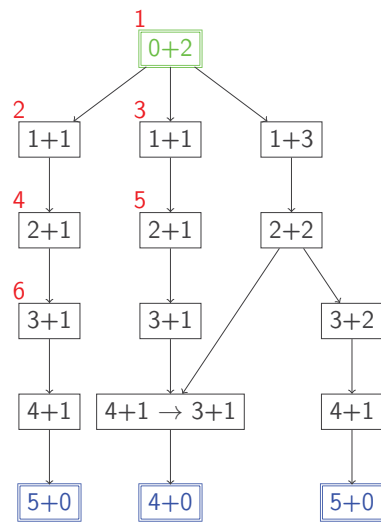
Abiertos

3+1
3+1
1+3

Cerrados

0+2
1+1
1+1
2+1
2+1

12



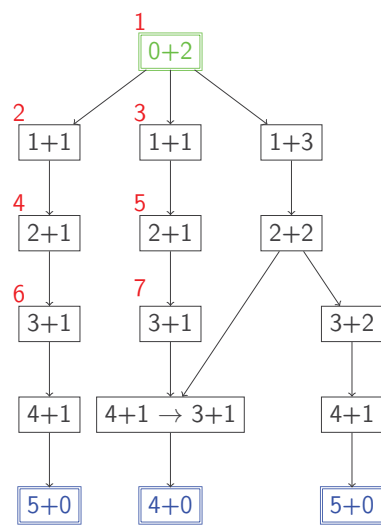
Abiertos

3+1
1+3
4+1

Cerrados

0+2
1+1
1+1
2+1
2+1
3+1

12



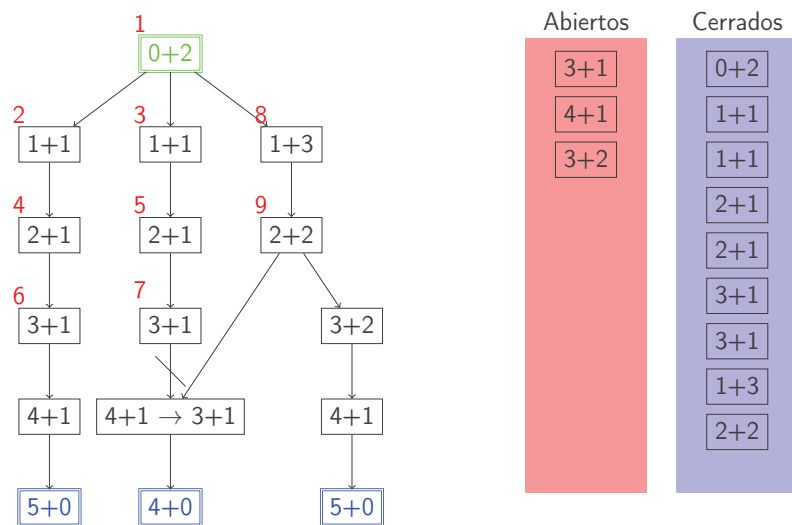
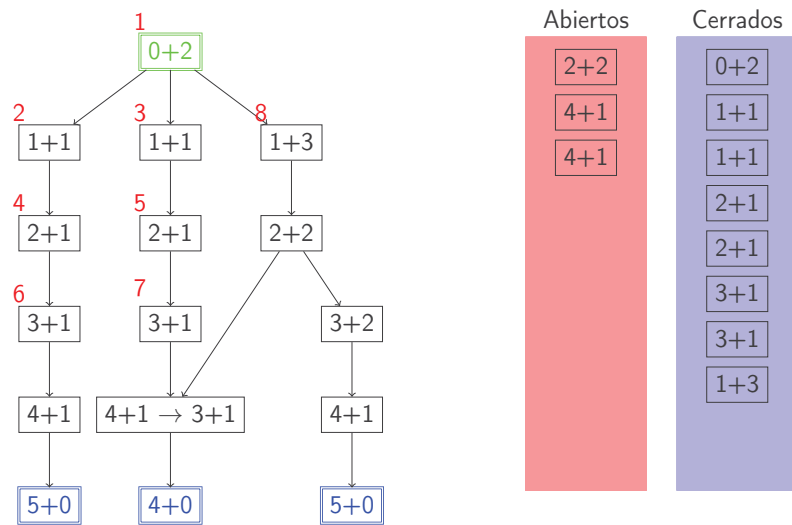
Abiertos

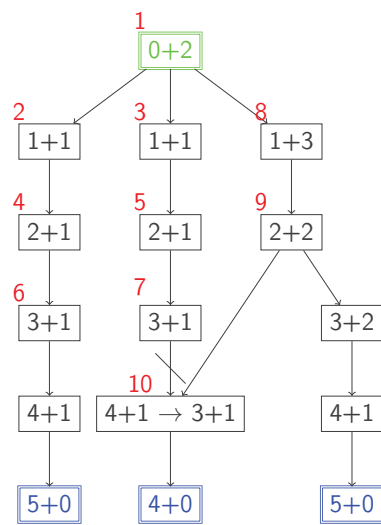
1+3
4+1
4+1

Cerrados

0+2
1+1
1+1
2+1
2+1
3+1
3+1

12





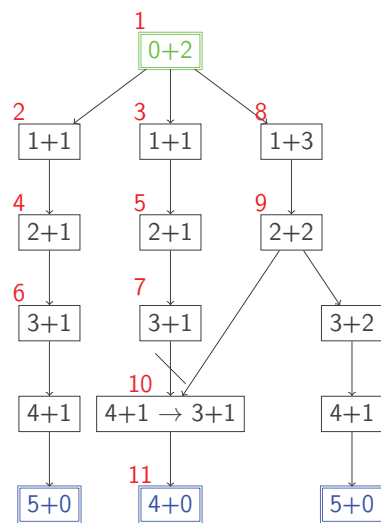
Abiertos

4+0
4+1
3+2

Cerrados

0+2
1+1
1+1
2+1
2+1
3+1
3+1
1+3
2+2
3+1

12



Abiertos

4+0
3+2
3+2

Cerrados

0+2
1+1
1+1
2+1
2+1
3+1
3+1
1+3
2+2
3+1
4+0

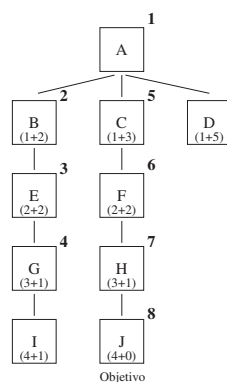
12

- ⦿ El algoritmo A* encontrará la solución óptima dependiendo del heurístico
- ⦿ Si el heurístico es admisible la **optimalidad** está asegurada
- ⦿ Un heurístico es admisible si se cumple la siguiente propiedad

$$\forall n \quad 0 \leq h(n) \leq h^*(n)$$

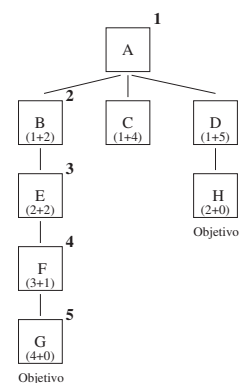
- ⦿ Por lo tanto, $h(n)$ ha de ser un **estimador optimista**, nunca ha de sobreestimar $h^*(n)$

13



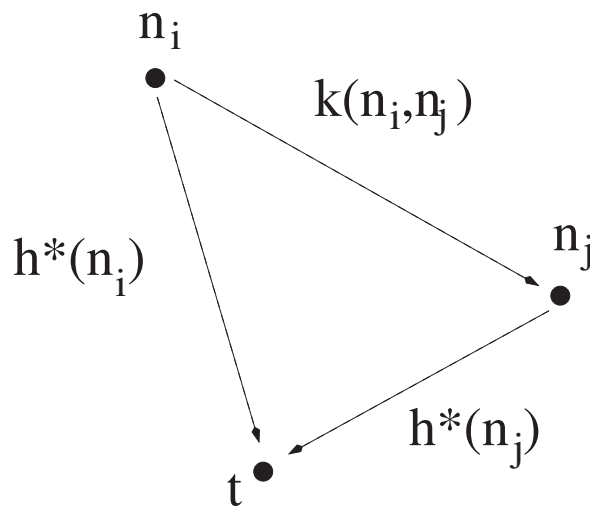
h subestima h^*

$h(B) = 2$, que no es real, hemos perdido tiempo pero acabaremos explorando otro camino (C)



h sobreestima h^*

si partiendo de D existiera un camino directo más corto nunca lo alcanzaríamos



- ⊙ $h^*(n_i)$ = coste mínimo desde n_i a t
- ⊙ $h^*(n_j)$ = coste mínimo desde n_j a t
- ⊙ $h^*(n)$ cumple desigualdad triangular

$$h^*(n_i) \leq k(n_i, n_j) + h^*(n_j)$$

- ⊙ La condición de consistencia exige que $h(n)$ se comporte como $h^*(n)$

$$h(n_i) - h(n_j) \leq k(n_i, n_j)$$

- ⊙ $h(n)$ consistente $\implies h(n)$ estimador uniforme de $h^*(n)$

15

- ⊙ $h(n)$ consistente $\implies g(n) = k(s, n)$ (camino óptimo a n)
- ⊙ Si $g(n) = k(s, n)$, dado que $h(n)$ siempre es constante $\implies f(n)$ es mínima
- ⊙ En este caso **no es necesario tratar los nodos duplicados cerrados** ya que los nodos expandidos ya no se podrán reexpandir (hemos llegado a ellos por el camino mínimo)

16

Dado un problema, existen tantos A^* para resolverlo como estimadores podamos definir.

Más informado

Para h_1 y h_2 admisibles, si se cumple

$$\forall n \neq final \quad 0 \leq h_2(n) < h_1(n) \leq h^*(n)$$

Entonces A_1^* es más informado que A_2^*

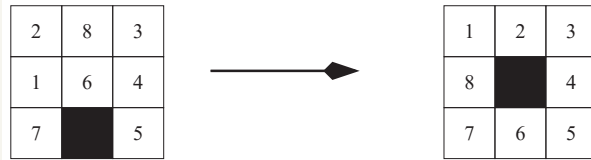
- ⊙ Si A_1^* es más informado que A_2^* entonces si el nodo n es expandido por $A_1^* \implies n$ es expandido por A_2^* (pero no al revés)
- ⊙ Eso quiere decir que A_1^* expande menor número de nodos que A_2^*

17

- ⊙ ¿Siempre elegiremos algoritmos más informados?
- ⊙ Compromiso entre:
 - Tiempo de cálculo de h
 - $h_1(n)$ requerirá más tiempo de cálculo que $h_2(n)$
 - Número de reexpansiones
 - A_1^* puede que reexpanda más nodos que A_2^*
 - Pero si A_1^* es consistente seguro que no lo hará
- ⊙ Pérdida de admisibilidad
 - Puede interesar trabajar con no admisibles para ganar rapidez
 - Algoritmos A_ϵ^* (ϵ -admisibilidad)

18

Ocho puzzle

 h_0

$h_0(n) = 0$ Equivalente a anchura prioritaria, h_0 admisible, muchas generaciones y expansiones

 h_1

$h_1(n) = \# \text{piezas mal colocadas}$ h_1 admisible, h_1 mas informado que h_0

19

 h_2

$h_2(n) = \sum_{i \in [1,8]} d_i$ $d_i \equiv$ distancia entre posición de la pieza i y su posición final

h_2 admisible, h_2 no más informado que h_1

$h_2([1, 2, 3, 4, 5, 6, 7, \emptyset, 8]) = h_1([1, 2, 3, 4, 5, 6, 7, \emptyset, 8]) = 1$

 h_3

$h_3(n) = \sum_{i \in [1,8]} d_i + 3 \cdot S(n)$; con $S(n) = \sum_{i \in [1,8]} s_i$

$s_i = \begin{cases} 0 & \text{si pieza } i \text{ no en el centro y sucesora correcta} \\ 1 & \text{si pieza } i \text{ en el centro} \\ 2 & \text{si pieza } i \text{ no en el centro y sucesora incorrecta} \end{cases}$

h_3 no es admisible, y por lo tanto no es más informado que h_2 , pero es más rápido

- ⊙ El algoritmo A* resuelve problemas en los que es necesario encontrar la mejor solución (el óptimo dentro de todo el espacio de estados)
- ⊙ Su coste en espacio y tiempo en el caso medio es mejor que los algoritmos de búsqueda ciega si el heurístico es adecuado
- ⊙ Existen problemas en los que la dimensión del espacio de búsqueda no permite su solución con A*
- ⊙ Además los nodos a almacenar por A* crecen exponencialmente si:

$$|h(n) - h^*(n)| \geq \log(h^*(n))$$

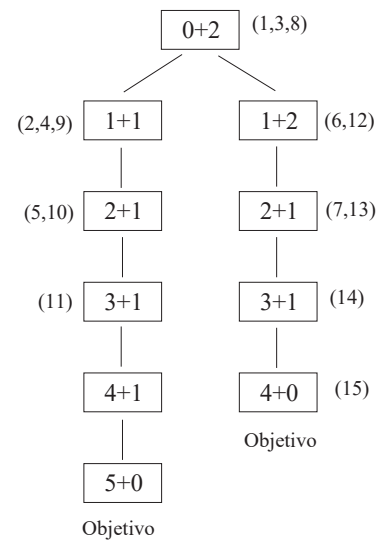
- ⊙ Conclusión: A* es un algoritmo que puede fácilmente consumir toda la memoria de un ordenador en problemas reales.

Busqueda Heurística con limitación de memoria

- ⊙ Similar a ID (es decir iteración de búsqueda en profundidad con un límite en la búsqueda)
- ⊙ En ID el límite lo daba una cota máxima en la profundidad
- ⊙ En IDA* el límite lo da una cota máxima sobre el valor de la función f

¡Ojo! La búsqueda es una DFS normal y corriente, el heurístico f sólo se utiliza para podar

Empezamos con $\text{corte} = f(\text{inicial})$



Algoritmo IDA*

Algoritmo: IDA* (límite entero)

$\text{prof} \leftarrow f(\text{Estado inicial})$

$\text{Actual} \leftarrow \text{Estado inicial}$

mientras no $\text{es_final?}(\text{Actual})$ **y** $\text{prof} < \text{limite}$ **hacer**

$\text{Est_abiertos.inicializa}()$

$\text{Est_abiertos.insertar}(\text{Estado inicial})$

$\text{Actual} \leftarrow \text{Est_abiertos.primer}()$

mientras no $\text{es_final?}(\text{Actual})$ **y** no $\text{Est_abiertos.vacia?}()$

hacer

$\text{Est_abiertos.borrar_primer}()$

$\text{Est_cerrados.insertar}(\text{Actual})$

$\text{Hijos} \leftarrow \text{generar_sucesores}(\text{Actual}, \text{prof})$

$\text{Hijos} \leftarrow \text{tratar_repetidos}(\text{Hijos}, \text{Est_cerrados}, \text{Est_abiertos})$

$\text{Est_abiertos.insertar}(\text{Hijos})$

$\text{Actual} \leftarrow \text{Est_abiertos.primer}()$

$\text{prof} \leftarrow \text{prof} + 1$

- ⊙ La función `generar_sucesores` solo genera aquellos con una f menor o igual a la del límite de la iteración
- ⊙ La estructura de abiertos es ahora una pila (búsqueda en profundidad)
- ⊙ Hemos de tener en cuenta que si tratamos los nodos repetidos el ahorro en espacio es nulo

- ⊙ Las reexpansiones de IDA* pueden suponer un elevado coste temporal
- ⊙ Existen algoritmos que por lo general reexpanden menos nodos: Memory Bound A* (MA*), Simplified Memory Bound A* (MA*).
- ⊙ Al igual que en el Best First Recursivo, su funcionamiento se basa en eliminar los nodos menos prometedores y guardar información que permita reexpandirlos:

- ⊙ Impone un límite de memoria (número de nodos que se pueden almacenar, mayor que $O(rp)$)
- ⊙ Explora usando A* y almacenando nodos mientras quepan en la memoria
- ⊙ Cuando no quepan eliminamos los peores guardando el mejor coste de los descendientes olvidados
- ⊙ Reexpandimos si los nodos olvidados son mejores
- ⊙ El algoritmo es completo si el camino solución cabe en memoria

Algoritmo Simplified Memory Bounded A* (SMA*)

Algoritmo: SMA* (límite entero)

Est_abiertos.insertar(Estado inicial)

mientras cierto **hacer**

```
    si Est_abiertos.vacia?() entonces
        retorna fallo
    Actual ← Est_abiertos.primer()
    si _final?(Actual) entonces
        retorna Actual
    Hijo ← generar_un_sucesor (Actual)
    si no es _final?(Hijo) y  $prof(Hijo) \equiv limite$  entonces
         $f(Hijo) \leftarrow \infty$ ;
    en otro caso
         $f(Hijo) \leftarrow \max(f(Actual), (g(Hijo) + h(Hijo)))$ ;
    si no hay _más_sucesores(Actual) entonces
        ajusta_f_a_minimo_sucesor (Actual)
    si todos _los_sucesores_abiertos(Actual, Est_abiertos) entonces
        Est_abiertos.borrar(Actual)
    si memoria_llena() entonces
        Eliminar_peor_nodo()
    Est_abiertos.insertar(Hijo)
```

- ⦿ Del nodo actual se genera un solo nodo cada vez con la función `generar_un_sucesor`
- ⦿ Cuando se han generado todos los sucesores de un nodo se ajusta su f a la del menor de sus sucesores
- ⦿ Cuando la memoria se llena se elimina el nodo con f mayor y, en caso de empate, de profundidad mayor